



Agentic AI System for Automotive Spare Parts Management

INTERNSHIP PROJECT BRIEF

Agentic AI System for Automotive Spare Parts Management

June – August 2026 | 12 Weeks |

Duration	June – August (approx. 12 weeks / 3 months)
Student Name & Profile	Mohnish Bhandarkar Final Year B.Tech
Domain	Manufacturing / Automotive Aftermarket
Tech Area	Agentic AI, Multimodal LLMs, Document Understanding
Mode	Part-time alongside regular semester coursework
Expected Output	Working prototype + technical report + demo video

1. Problem Statement

Automotive service centers, garages, and dealerships regularly request specific spare parts for vehicle repairs. A spare parts factory holds inventory across hundreds of assemblies like steering systems, seat assemblies, braking systems, and more, but tracking what is available, what is running low, and what needs to be reordered is largely manual today.

Assembly information is typically locked inside PDF service brochures and exploded-view diagrams. A technician at a service center sends in a request like:

“I need all parts for the steering assembly of a Maruti Swift”

Today, someone manually cross-references the brochure, checks the warehouse system, and raises a flag if anything is short. This project asks: can an agentic AI system do this end-to-end, intelligently?

2. Project Objective

Build a multi-agent AI system that:

- Reads a mechanical assembly brochure (PDF or scanned document) and extracts the complete Bill of Materials (BOM) for a requested assembly
- Understands exploded-view assembly diagrams using a vision-capable AI model to identify parts and part numbers
- Queries a simulated spare parts inventory database to check stock levels
- Flags parts that are unavailable or below a defined threshold
- Generates a structured fulfilment report for the service centre request

The student is encouraged to identify one additional agent or capability that makes the system more useful, this is the creative design space.

3. High-Level System Architecture

The system is envisioned as a pipeline of specialised agents, each handling one job. How they communicate, what data format they use, and how errors are handled is left to the student to design.

Agent	Responsibility	Notes for Student
Document Agent	Parse PDF brochures; extract BOM text	<i>Consider how to handle multi-column layouts and tables in PDFs</i>
Vision Agent	Read exploded-view diagrams; identify parts visually	<i>This is the most technically challenging piece, explore multimodal models</i>
Inventory Agent	Query a mock inventory DB; return stock levels	<i>Design your own DB schema; think about what fields matter for spare parts</i>
Decision Agent	Compare BOM vs. stock; flag gaps	<i>Define what 'below threshold' means, is it fixed or dynamic?</i>
Report Agent	Produce a structured output for the service center	<i>Format matters, who reads this output and how?</i>

[Your Agent]	Student's creative addition	<i>What would make this system genuinely more useful in a real factory?</i>
---------------------	-----------------------------	---

The student should pick something they find genuinely interesting. The quality of reasoning behind the choice matters as much as the implementation.

4. Weekly Milestones

Milestones are structured around a part-time schedule. Expect to spend roughly 10–15 hours per week on the project. Weeks with exams or submissions can be lighter.

Week	Dates	Focus	Deliverable / Checkpoint
W 1–3	<i>Jun 01–21</i>	Orientation & Setup	Read 2–3 papers on agentic AI & document parsing. Set up dev environment. Gather 2–3 sample brochures/BOMs. Write a 1-page problem framing document.
W 3–4	<i>Jun 22–Jul 5</i>	Document Agent	Working agent that reads a PDF brochure and outputs a structured BOM list. Handle at least one text-heavy and one table-format brochure.
W 5	<i>Jul 6–12</i>	Vision Agent	Agent that takes an exploded-view diagram image and identifies parts. Even partial extraction is a good result here, document what works and what doesn't.
W 6	<i>Jul 13–19</i>	Inventory Agent + DB	Design and populate a mock inventory database. Agent queries it given a BOM and returns stock status. Threshold logic implemented.
W 7	<i>Jul 20–26</i>	Decision + Report Agent	End-to-end pipeline works for one assembly type. Report generated correctly with flags. Mid-point review with mentor.
W 8	<i>Jul 27–Aug 2</i>	Creative Agent / Extension	Student designs and builds their own addition to the system. Document the design rationale, why this agent, what does it change?

W 9	<i>Aug 3–16</i>	Integration & Testing	All agents work together. Test with 3–5 different assembly requests. Log failure cases honestly. Refine based on findings.
W 10	<i>Aug 17–30</i>	Documentation & Demo	Final technical report. Demo video (5–10 min). Code pushed to GitHub with README. Submit by August 30.

5. Suggested Tech Stack

These are starting points, not mandates. If the student finds better tools, use them, just document the reasoning.

- Language: Python 3.10+
- LLM / Vision: GPT-4o, Claude 3, or Gemini 1.5 (any API with vision support)
- PDF Parsing: PyMuPDF, pdfplumber, or LlamaParse
- Agentic Framework: LangChain, LangGraph, CrewAI, or AutoGen, student's choice
- Inventory DB: SQLite or a simple JSON flat file is sufficient
- Report Output: Markdown, PDF, or a simple web dashboard

6. Deliverables

Deliverable	Description
Working Prototype	GitHub repo with clean, commented code and a setup README
Technical Report	8–12 pages covering problem, architecture, agent design, results, and limitations
Demo Video	5–10 minute walkthrough showing the system handling a real assembly request end-to-end
Mid-point Check-in	Brief slide deck (week 7), what's working, what's not, and the creative agent plan
Failure Log	Honest documentation of what the system gets wrong and why

8. Evaluation Criteria

- Does the core pipeline work reliably end-to-end? (40%)
- Quality and originality of the student's creative agent or extension (25%)
- Clarity of the technical report and code documentation (20%)
- Honest assessment of limitations and what could be improved (15%)

A system that works for 3 assemblies and honestly documents why it fails on the 4th is far more valuable than one that claims to work for everything.

9. A Note to the Student

This project sits at an interesting crossroads: you know how mechanical assemblies actually work, what an exploded-view drawing communicates, and why parts fail together. Most AI engineers don't. That domain knowledge is your real advantage, use it to make design decisions that a pure software person wouldn't think to make.

The agentic AI tooling is new and moves fast. Don't get stuck trying to master every framework. Pick one, go deep, and build something that actually works on a real problem. A focused, well-documented prototype beats an ambitious half-finished system every time.

Good luck and enjoy the build.